

Figure 1: Real-time connectivity

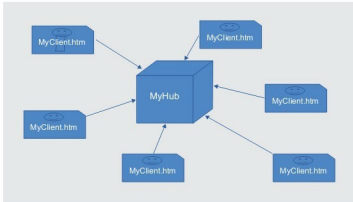


Figure 2: How SignalR works

Manager Console

2. Run the command `install-package Microsoft.AspNet.SignalR`

In SignalR, not only can the client contact the server, but the server can also contact the client. These aren't just simple HTTP responses, but actual method calls from server to client, like push technology. Clients can even contact other clients through the SignalR server-side component. All this is possible because SignalR creates a persistent connection. Thus, live communication can easily be achieved through SignalR.

Figure 2 shows one hub (server), MyHub, with six clients, MyClient.htm, which are connected to MyHub. If one of the clients sends a message 'M' to MyHub, then the server can send an acknowledgement to the particular client that the server has received the message in the traditional way; this is achieved through 'Ajax polling'.

With SignalR, the hub/server will send the message 'M' to all the clients that are connected to the hub/server instead of sending it to the one which has sent the message. In this way, the message is broadcast to all the clients that are connected to the hub. However, we can also restrict broadcasting by specifying which clients should receive the message from the hub/server. Though the same can be achieved through Ajax polling or Web sockets, in the latter case, you have to include all the work that SignalR has already completed.

Features provided by SignalR

- It supports third party security providers such as Microsoft

Live, OpenAuth, Google, Facebook and Twitter.

- It's also important to remember that clients of SignalR applications aren't restricted to just Web browsers.
- SignalR provides client libraries for .NET, JavaScript, Silverlight, Windows Phone, Windows RT, and even iOS and Android through Xamarin.

Hosting of SignalR

There are three options available for hosting SignalR:

1. Self-hosting in Console application
2. IIS-hosting
3. Self-hosting in Windows service

Hosting is done using the self-host library, which is built on OWIN (Open Web Interface for .NET). OWIN defines an abstraction between .NET Web servers and Web applications. It decouples Web applications from the server, which makes OWIN ideal for self-hosting a Web application in your own process, outside of IIS.

Advantages of self-hosting over IIS-hosting

1. Self-hosting becomes better than integrated clients in environments where IIS is neither available nor desirable.
2. If one wants to avoid the performance overhead of IIS, then self-hosting is the better choice.
3. The service hosting option is ideal when you want the hub to be always on, and can be restarted automatically when the machine starts.
4. To keep things simple, self-hosting in a console application is the better choice.

SignalR architecture

SignalR provides high level APIs to make remote procedure calls, which can invoke client functions and vice versa to maintain communication between the client and server. Figure 3 shows high level architectural components of SignalR.

SignalR primarily makes use of the Hubs API and the Persistent API for communication.

Hubs API: The Hubs API helps to make remote procedure calls from the server to connected clients and from clients to the server. In server code, you define methods that can be

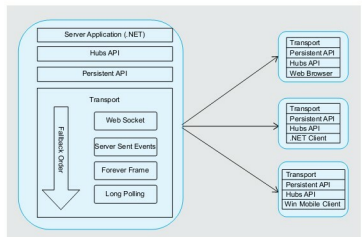


Figure 3: SignalR architecture